

CS 305: THEORY OF COMPUTATION — WEEK 2 LECTURE

Deterministic Finite Automata (DFA)

Formal 5-Tuples, State Diagrams, Product Construction & Line-Rate Packet Filtering

School of Technology

Ghana Institute of Management and Public Administration (GIMPA)

Textbook Reference: Michael Sipser, *Introduction to the Theory of Computation* (3rd Ed.), Section 1.1

Week 1 Recap & Week 2 Roadmap

Transitioning from informal mathematical foundations to rigorous machine models

Week 1 Foundations Recap

What We Established:

- **Alphabets (Σ):** Non-empty finite sets of symbols.
- **Strings ($w \in \Sigma^*$):** Finite sequences of symbols, length $|w|$, empty string ϵ .
- **Languages ($L \subseteq \Sigma^*$):** Formal decision problems.
- **Chomsky Type 3:** The class of regular languages recognizes patterns with $\mathcal{O}(1)$ constant memory.

Today's Agenda (Week 2)

Core Topics Today:

- **Formal 5-Tuple Definition:** $M = (Q, \Sigma, \delta, q_0, F)$.
- **DFA Design Patterns:** Substring search, modulo counting, and binary divisibility.
- **Formal Computation & Extended δ^* :** Exact acceptance conditions.
- **Closure Proofs:** Product construction for Union ($\cup$) and Intersection ($\cap$).
- **Systems Spotlight:** Line-rate 100 Gbps Deep Packet Inspection (Snort/Zeek).

Pedagogical Milestone: Today we assign **Problem Set 1** (due Week 3), focusing on formal DFA constructions and closure property proofs.

Formal Definition of a DFA

Michael Sipser, Definition 1.5: The 5-tuple mathematical foundation

The 5-Tuple Specification

Definition 2.1: DFA 5-Tuple

A **Deterministic Finite Automaton** (DFA) is a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

1. Q is a finite set of **states**.
2. Σ is a finite set called the **alphabet**.
3. $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**.
4. $q_0 \in Q$ is the **start (initial) state**.
5. $F \subseteq Q$ is the set of **accept (final) states**.

What Makes It “Deterministic”?

Crucial Deterministic Invariants

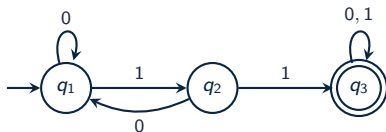
- **Total Function:** For every state $q \in Q$ and every symbol $a \in \Sigma$, $\delta(q, a)$ must specify **exactly one** next state.
- **No Choices / No Guessing:** The machine's path on any input string w is uniquely determined.
- **No ϵ -Transitions:** The DFA cannot change state without consuming an input symbol from Σ .

State Meaning: Each state in Q represents an **equivalence class of past inputs** that require identical future treatment.

Representing DFAs: Diagrams vs. Transition Tables

Two isomorphic representations of the transition function δ

Visual State Diagram



State Semantics:

- q_1 : Have not seen a 1 / last symbol was 0.
- q_2 : Just read a single 1.
- q_3 : Have seen substring 11 (Accepting Sink).

Formal Transition Table (δ)

State $q \in Q$	Input 0	Input 1
$\rightarrow q_1$	q_1	q_2
q_2	q_1	q_3
$*q_3$	q_3	q_3

Formal Specification of M_1 :

$$M_1 = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_3\})$$

Language Recognized:

$$L(M_1) = \{w \in \{0, 1\}^* \mid w \text{ contains substring } 11\}$$

DFA Design Pattern 1: Tracking Substrings

Methodology: Building progress chains with corrective fallback transitions

Design Objective: Substring "010"

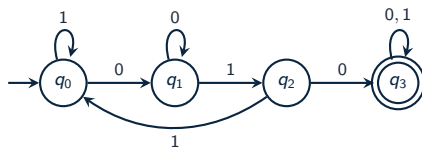
Target Language:

$L = \{w \in \{0, 1\}^* \mid w \text{ contains substring } 010\}$

State Meaning Formulation:

- q_0 : Matched prefix ε (no progress).
- q_1 : Matched prefix 0.
- q_2 : Matched prefix 01.
- q_3 : Matched 010 (Accepting trap state).

State Diagram with Fallback Arcs



Crucial Fallback Transition Analysis:

If in q_2 (seen 01) we read 1 $\implies$ string ends in 011. Longest prefix of 010 in suffix is $\varepsilon \implies$ fallback to q_0 !

Pattern Rule: On mismatch, transition to the state representing the **longest prefix** of the target pattern that matches the current suffix.

DFA Design Pattern 2: Parity & Modulo Counting

Tracking independent modular invariants via Cartesian state product

Design Objective: Dual Parity

Target Language:

$L = \{w \in \{0, 1\}^* \mid w \text{ has even 0s and odd 1s}\}$

Decomposing Invariants:

- Number of 0s mod 2: {Even, Odd} (2 states).
- Number of 1s mod 2: {Even, Odd} (2 states).
- Total States: $2 \times 2 = 4$ composite states.

State Labeling:

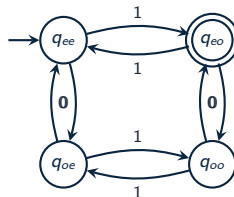
q_{ee} : (Even 0s, Even 1s) — Start State

q_{eo} : (Even 0s, Odd 1s) — **Accept State**

q_{oe} : (Odd 0s, Even 1s)

q_{oo} : (Odd 0s, Odd 1s)

4-State Parity Diamond



Modular Memory Rule: Any modulo counting (mod k) requires exactly k states. A DFA can count (mod k) for any fixed k , but cannot count to infinity.

DFA Design Pattern 3: Binary Divisibility by 3

Arithmetic stream processing: Updating numerical values on the fly

Mathematical Invariant

Language Specification:

$L_{\text{div}3} = \{w \in \{0,1\}^* \mid \text{val}(w) \text{ is divisible by } 3\}$

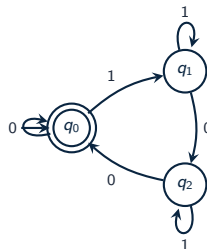
Examples in L : 0 (0), 11 (3), 110 (6), 1001 (9).

Value Recurrence Relation: When appending bit $b \in \{0,1\}$ to binary string w :

$$\text{val}(wb) \equiv 2 \cdot \text{val}(w) + b \pmod{3}$$

$$\delta(q_r, b) = q_{(2r+b) \bmod 3}$$

3-State Remainder Machine



State Transition Logic:

State q_r tracks remainder $r \in \{0,1,2\}$:

From q_1 : on 0 $\rightarrow (2 \cdot 1 + 0) \bmod 3 = 2 \Rightarrow q_2$.

From q_1 : on 1 $\rightarrow (2 \cdot 1 + 1) \bmod 3 = 0 \Rightarrow q_0$.

Formal Definition of Computation & Acceptance

Michael Sipser, Definition 1.5: How a DFA formally processes strings

Formal Acceptance Criteria

Definition 2.2: Computation Sequence

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. Let $w = w_1 w_2 \dots w_n$ ($w_i \in \Sigma$).

M **accepts** w if there is a state sequence $r_0, \dots, r_n \in Q$ where:

1. $r_0 = q_0$ (Initial state)
2. $r_i = \delta(r_{i-1}, w_i)$ for $i = 1, \dots, n$ (Transitions)
3. $r_n \in F$ (Accept state)

Language of an Automaton

Recognizing a Language:

M **recognizes** language A if:

$$A = \{w \mid M \text{ accepts } w\}$$

We write:

$$L(M) = A$$

Definition 2.3: Regular Language

A language L is called **regular** if some DFA recognizes it:

$$\text{Regular} = \{L \mid \exists \text{ DFA } M \text{ with } L(M) = L\}$$

Important Distinction: An automaton M may *accept* many individual strings, but it *recognizes* only **one** language: the set $L(M)$ of all accepted strings.

Extended Transition Function (δ^*)

Extending the single-symbol transition function δ to entire strings $w \in \Sigma^*$

Inductive Definition of δ^*

Definition 2.4: Extended δ^*

Define $\delta^* : Q \times \Sigma^* \rightarrow Q$ inductively by:

$$\text{Base Case: } \delta^*(q, \varepsilon) = q$$

$$\text{Inductive Step: } \delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

for any state $q \in Q$, string $w \in \Sigma^*$, and symbol $a \in \Sigma$.

String Concatenation Property:

$$\delta^*(q, uv) = \delta^*(\delta^*(q, u), v)$$

Compact Formal Acceptance

Direct Language Membership:

Using δ^* , language acceptance collapses into a single elegant expression:

$$w \in L(M) \iff \delta^*(q_0, w) \in F$$

Rejection Condition:

$$w \notin L(M) \iff \delta^*(q_0, w) \in Q \setminus F$$

Why δ^* is Essential:

δ^* is the foundational tool for proving correctness of automata constructions by induction on the length of input strings $|w|$.

In-Class Check-in 2.1: DFA Traces & State Reasoning

Test your mastery of state transitions, invariants, and memory limits

Check-in Questions

Problem 2.1A: Trace Evaluation

Consider the Divisible-by-3 DFA $M_{\text{div}3}$ from Slide 7. Trace the exact sequence of states on input $w = 1101_2$ (13_{10}):

1. What is the state sequence r_0, r_1, r_2, r_3, r_4 ?
2. Does $M_{\text{div}3}$ accept 1101?

Problem 2.1B: Conceptual Limit

Can a 100-state DFA recognize the language $L = \{a^n b^n \mid n \geq 0\}$ of matching brackets?

In-Class Workspace & Discussion

Trace Strategy & State Analysis:

- **Step-by-Step Transition:** Begin at start state $r_0 = q_0$. Compute next state $r_i = \delta(r_{i-1}, w_i)$ for each incoming bit $w_i \in \{1, 1, 0, 1\}$.
- **Acceptance Criterion:** Check whether terminal state $r_4 \in F$. Compare against arithmetic remainder 13 (mod 3).
- **Pigeonhole Insight:** If a DFA has k states, what happens if a language requires distinguishing an unbounded number of prefix lengths?
- **Collaborative Action:** Sketch the state path $(r_0 \rightarrow r_1 \cdots \rightarrow r_4)$ and verify with your peer.

[Active learning drill — solutions analyzed interactively in lecture.]

Closure Properties: Union ($\cup$) and Product Construction

Michael Sipser, Theorem 1.25: Class of regular languages is closed under union

Theorem & Proof Strategy

Theorem 2.1: Closure Under Union

If A_1 and A_2 are regular languages, then $A_1 \cup A_2$ is also regular.

Proof Strategy (Parallel Simulation):

Let DFA $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 , and DFA $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 .

Simulate both machines in parallel on input w using the **Cartesian product** of states:

$$Q = Q_1 \times Q_2$$

Formal Construction of M

Construct $M = (Q, \Sigma, \delta, q_0, F)$:

1. $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1, r_2 \in Q_2\}$.
2. Alphabet Σ is identical.
3. $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.
4. Start State: $q_0 = (q_1, q_2)$.
5. Accept States:

$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

Accept iff *either* M_1 accepts or M_2 accepts.

State Space Size: If M_1 has m states and M_2 has n states, the product DFA has exactly $|Q| = m \cdot n$ states.

Closure Properties: Intersection ($\cap$) & Complement ($\bar{L}$)

Algebraic robustness of regular languages under boolean operations

Closure Under Complement ($\bar{L}$)

Theorem 2.2: Complement Closure

If L is regular, then $\bar{L} = \Sigma^* \setminus L$ is regular.

Construction: Let $M = (Q, \Sigma, \delta, q_0, F)$ recognize L .
Construct:

$$M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$$

Mechanism: Invert accept states! All accept states become non-accept, and vice versa.

Closure Under Intersection ($\cap$)

Theorem 2.3: Intersection Closure

If A_1 and A_2 are regular, then $A_1 \cap A_2$ is regular.

Method 1 (Product Construction): Use the exact same product machine $Q_1 \times Q_2$, but set:

$$F = F_1 \times F_2 = \{(r_1, r_2) \mid r_1 \in F_1 \text{ and } r_2 \in F_2\}$$

Method 2 (De Morgan's Laws):

$$A_1 \cap A_2 = \overline{\overline{A_1} \cup \overline{A_2}}$$

Algebraic Summary: The class of regular languages forms a **Boolean Algebra** closed under Union ($\cup$), Intersection ($\cap$), and Complement ($\bar{L}$).

Systems Spotlight: Line-Rate Deep Packet Inspection (DPI)

High-performance network security in Snort, Zeek, and hardware ASICs

The 100 Gbps Real-Time Challenge

The Engineering Dilemma:

Network intrusion detection systems (NIDS) must scan packet payloads against 10,000+ attack regex signatures in real-time.

- At 100 Gbps, one byte arrives every **80 picoseconds**.
- Packet drops or buffering are unacceptable.

Why DFAs Win:

A DFA guarantees $\mathcal{O}(1)$ transition time per byte:
`next_state = table[current_state][byte]`. Processing time is strictly independent of payload contents!

State Explosion & D^2 FA Innovation

ACM SIGCOMM Research Anchor:

Kumar et al., “Algorithms to Accelerate Multiple Regular Expressions Matching for Deep Packet Inspection” (ACM SIGCOMM).

The State Space Explosion Problem: Combining 1,000 regex rules into one DFA yields millions of states $\implies$ cannot fit in fast CPU L1/L2 cache!

Deferred DFA (D^2 FA) Solution: Compress redundant state transitions using default back-pointers, reducing memory by **95%** while retaining deterministic speed.

Compilers Spotlight: Lexical Analyzers (Flex / LLVM)

How industrial compilers convert character streams to AST tokens in $\mathcal{O}(n)$ time

The Lexer DFA Pipeline

From Source Code to Tokens:

Compiler lexers convert characters into tokens:

"while (x <= 100)" $\longrightarrow$

[WHILE, LPAREN, ID(x), LEQ, NUM(100), RPAREN]

Mechanism:

- Rules defined as regex: `while`, `[a-zA-Z_][a-zA-Z0-9_]*`, `[0-9]+`.
- Flex compiles all regexes into a single combined DFA!

Maximum Munch (Longest Match)

Disambiguation Rule:

If input is "while_count", is it the keyword `while` or identifier `while_count`?

DFA Solution: The DFA continues reading until no valid transition exists, then rolls back to the **most recent accept state**. This guarantees $\mathcal{O}(n)$ tokenization!

Compiler Impact: Every programming language compiler you use relies on DFA execution for its front-end lexical tokenization.

Preview of Week 3: Nondeterministic Finite Automata (NFA)

Relaxing determinism: Exploring multiple computational paths simultaneously

Why Nondeterminism?

The Limits of DFA Simplicity:

Designing DFAs directly for languages like:

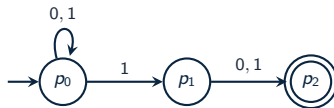
$$L = \{w \in \{0,1\}^* \mid \text{the 5th symbol from the end is 1}\}$$

requires $2^5 = 32$ states!

The NFA Advantage:

- Allows **zero, one, or multiple transitions** on the same input symbol.
- Introduces **ϵ -transitions** (spontaneous jumps without reading input).
- Reduces state complexity exponentially during design.

Fundamental Equivalence Ahead



Equivalence Theorem Preview:

Rabin & Scott (Turing Award): Every NFA can be converted to an equivalent DFA via the **Powerset Construction**:

$$L(\text{NFA}) \equiv L(\text{DFA})$$

Week 2 Synthesis & Problem Set 1

Mastery checklist, required readings, and milestone deliverables

Week 2 Concept Checklist

You should now be able to:

- Write formal 5-tuples $M = (Q, \Sigma, \delta, q_0, F)$ without ambiguity.
- Construct state diagrams for substring, modulo, and divisibility patterns.
- Formulate computation traces and state invariants using extended δ^* .
- Execute the Product Construction proof for Union ($\cup$) and Intersection ($\cap$).
- Explain the $\mathcal{O}(1)$ throughput advantages of DFAs in Zeek/Snort DPI and compiler lexers.

Milestone: Problem Set 1 Assigned

Problem Set 1 (Due Week 3):

- **Problem 1:** Formal DFA design for binary numbers divisible by 5.
- **Problem 2:** Dual parity DFA for 3-letter alphabet $\{a, b, c\}$.
- **Problem 3:** Formal proof that regular languages are closed under Symmetric Difference ($A \oplus B$).

Required Reading:

Sipser (3rd Ed.), **Section 1.1: Finite Automata** (pp. 31–44).

Office Hours: Tuesdays & Thursdays 14:00–16:00 GMT | GIMPA School of Technology